Домашнее задание 5. Firebase: вход и каталог в Firestore

Каталог из ДЗ 4 переносим на Firebase: вход по email и паролю, данные в Cloud Firestore. Подключение — по гайду «Firebase: подключаем к Flutter-проекту» в LMS.

Дедлайн: понедельник, 26 октября, 23:59 по МСК.

1. Что нужно сделать

Часть 1. Подключение

1. Проект Firebase подключён командой `flutterfire configure --project=<id> --platforms=android,web`.
2. Пакеты: `firebase_core`, `firebase_auth`, `cloud_firestore`.

Часть 2. Вход

1. Экран входа: «Email», «Пароль», кнопки «Войти» и «Зарегистрироваться».
2. Без входа каталог недоступен. Кнопка «Выйти» возвращает на экран входа.
3. При ошибке входа на экране виден `e.code`.
4. Регистрация — только тестовыми почтами (`test1@test.com`).

Часть 3. Каталог в Firestore

1. Правило Firestore — из подсказок, вместо тестового.
2. Коллекция — тема каталога латиницей во множественном числе: `books`, `dogs`, `movies`.
3. Каталог читается через `snapshots()` и `StreamBuilder`. MockAPI не используется.
4. В модели есть `fromFirestore` (id из `doc.id`) и `toJson`.
5. Загрузка — `CircularProgressIndicator`, ошибка — текст ошибки.

Часть 4. Добавление

1. Форма из ДЗ 4 сохраняет элемент через **add**. Он появляется в списке сам.
2. В коллекции 8 и более элементов, добавленных формой.

Лайки и фильтр «Только избранные» — локально, как в ДЗ 4.

По желанию: лайки в `users/{uid}/favorites/{id элемента}` — у каждого пользователя свои.

Подсказки:

- Правило: Firestore Database → Rules, вставить и нажать Publish:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

- Добавление:

```
await FirebaseFirestore.instance
    .collection('books')
    .add(book.toJson());
```

- Поток — поле `State`, не внутри `build`:

```
final _itemsStream = FirebaseFirestore.instance
    .collection('books')
    .snapshots();
```

- Модель:

```
factory Book.fromFirestore(
    DocumentSnapshot<Map<String, dynamic>> doc) {
  final data = doc.data()!;
  return Book(
    id: doc.id,
    title: data['title'] as String,
  );
}
```

- По желанию, лайк:

```
final uid = FirebaseAuth.instance.currentUser!.uid;
final fav = FirebaseFirestore.instance
    .collection('users').doc(uid)
    .collection('favorites').doc(itemId);
if (isLiked) {
  await fav.delete();
} else {
  await fav.set({'liked': true});
}
```

- `permission-denied` — не выполнен вход или не опубликовано правило.
- Ошибки установки и сборки — в гайде, раздел «Если что-то пошло не так».

2. Как сдать работу через GitHub

Шаг 1. Смерджите hw4

Pull Request по ДЗ 4 → Merge pull request → Confirm merge.

Шаг 2. Создайте ветку hw5

```
git checkout main
git pull origin main
git checkout -b hw5
```

Шаг 3. Настройте .gitignore

Файлы от `flutterfire configure` (`firebase_options.dart`, `google-services.json`, `firebase.json`) коммитим. В `.gitignore` добавьте:

```
# кэш Hosting
.firebase/

# логи CLI
firebase-debug.log

# ключи сервисного аккаунта
*-firebase-adminsdk-*.json
```

Шаг 4. Закоммитьте и отправьте

```
git add .
git commit -m "hw5: firebase auth and firestore"
git push -u origin hw5
```

Шаг 5. Откройте Pull Request в main

Compare & pull request → Create pull request. Не мерджите.

Шаг 6. Запишите видео, 40–60 секунд

- вход — каталог из Firestore;
- добавление — элемент в списке и в консоли Firebase;
- выход.

Шаг 7. Сдайте в LMS

Видео и ссылка на открытый Pull Request.

3. Важно

- Репозиторий публичный.

- Письмо GitHub **Possible valid secrets detected** про ключ в `firebase_options.dart` — не секрет. **Security → Secret scanning** → закрыть как **False positive**.
- JSON сервисного аккаунта не коммитить никогда.
- Ошибка — не пересоздавайте проект, пришлите текст ошибки целиком.